2303A51319

A.Anjali

LAB ASSIGNMENT-9.3

Task-1:

Prompt: Generate a Google Style docstring for a Python function that calculates

the sum of even numbers and the sum of odd numbers from a list of integers.

The docstring should include a short description, Args, Returns,

and Raises sections. Ensure the documentation is clear and concise.

Analysis: The manual docstring is more detailed and includes error handling, making it more complete. The AI-generated docstring is clear and correctly formatted but may miss edge cases like type validation. AI is useful for quick documentation, but manual refinement improves clarity and completeness.

task 1.py > ...

```python
def sum_even_odd(numbers):
    """
    Calculate the sum of even and odd numbers in a list.

    Args:
        numbers (list): A list of integers to process.

    Returns:
        tuple: A tuple containing (sum_of_evens, sum_of_odds).

    Example:
        >>> sum_even_odd([1, 2, 3, 4, 5, 6])
        (12, 9)
    """
    sum_even = sum(num for num in numbers if num % 2 == 0)
    sum_odd = sum(num for num in numbers if num % 2 != 0)
    return sum_even, sum_odd


# Test cases
if __name__ == "__main__":
    test_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    even_sum, odd_sum = sum_even_odd(test_list)
    print(f"Even sum: {even_sum}")
```

```
Enter a list of integers separated by commas: 1,2,3
Even sum: 2
Odd sum: 4
```

Task-2:

Prompt: Add inline comments to the following Python class.

Explain each logical block and important operation in simple language.

Avoid redundant comments and focus on improving readability for new developers.

Analysis: Manual comments explain both what the code does and why it does it, improving understanding. AI-generated comments are accurate but often basic and sometimes redundant. AI helps speed up documentation, but developers must review comments for meaningful context.

task2.py > ...

```python
# Student Management Module - Class Definition
class sru_student:
    # Constructor to initialize student object with basic details
    def __init__(self, name, roll_no, hostel_status):
        # Store student's full name
        self.name = name
        # Store unique roll number for identification
        self.roll_no = roll_no
        # Store hostel accommodation status (True/False)
        self.hostel_status = hostel_status
        # Initialize fee amount to zero
        self.fee_amount = 0

    # Method to update student fee
    def fee_update(self, amount):
        # Add the given amount to existing fee
        self.fee_amount += amount
        # Confirm fee update with console message
        print(f"Fee updated for {self.name}. Total fee: Rs.{self.fee_amount}")

    # Method to display all student details
    def display_details(self):
        # Print student's name
        print(f"Name: {self.name}")
```

```
Enter student's full name: palomi
Enter student's roll number: 45
Does the student have hostel accommodation? (yes/no): yes
Enter fee amount to update: 50000
Fee updated for palomi. Total fee: Rs.50000.0
Name: palomi
Roll No: 45
Hostel Status: Yes
Fee Amount: Rs.50000.0
```

Task 3:

Prompt: Generate a NumPy-style module-level docstring and function-level

docstrings for a Python calculator module containing functions:

add, subtract, multiply, and divide.

Include sections for Parameters, Returns, and Raises where applicable.

Ensure the documentation follows structured scientific documentation style.

Analysis: Manual NumPy-style documentation follows strict structure and includes exceptions clearly. AI-generated documentation is readable and well-formatted but may lack detailed explanations or special case handling. AI is effective for generating structured drafts, but manual review ensures accuracy and professional quality.

Untitled-1.py

C: > Users > anjal > OneDrive > Desktop > Untitled-1.py > ...

```
154    def get_user_input():
176            print("4. Divide (/)")
177
178            operation = input("\nEnter operation (1/2/3/4): ").strip()
179
180            # Get second number
181            num2 = float(input("Enter second number: "))
182
183            # Perform calculation
184            if operation == "1":
185                result = add(num1, num2)
186                print(f"\n{num1} + {num2} = {result}")
187            elif operation == "2":
188                result = subtract(num1, num2)
189                print(f"\n{num1} - {num2} = {result}")
190            elif operation == "3":
191                result = multiply(num1, num2)
192                print(f"\n{num1} * {num2} = {result}")
193            elif operation == "4":
194                result = divide(num1, num2)
195                print(f"\n{num1} / {num2} = {result}")
196            else:
197                print("\nInvalid operation selected.")
```

Output:

PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS GITLENS

```
PS C:\Users\anjal\OneDrive\Desktop\AI_Assistent_lab> 3.4
3.4
PS C:\Users\anjal\OneDrive\Desktop\AI_Assistent_lab> 1
1
PS C:\Users\anjal\OneDrive\Desktop\AI_Assistent_lab> 3.5
3.5
PS C:\Users\anjal\OneDrive\Desktop\AI_Assistent_lab>
```